

CMSC 471 Programming Assignment 1

Zachary Barry

September 2026

Section: 2

1 Question 1: Heuristic Design and Admissibility

For steps, $h(n)$ is the number of positions where the current word and goal differ. For example, if the current word is dog and the goal is cat, all three positions are different, so the heuristic is 3.

This is a lower bound because one action can only change one letter. If there are k incorrect positions, then at least k actions are needed to make all of those positions match the goal. Since every action has a cost of 1, the remaining cost must be at least k . Therefore, this heuristic cannot overestimate the optimal remaining cost.

For Scrabble, $h(n)$ is the sum of the Scrabble values of the goal letters at each mismatched position. For example, if the current word is dog and the goal is cat, the heuristic is the value of c plus the value of a plus the value of t, which is $3 + 1 + 1 = 5$.

This is a lower bound because every incorrect position eventually has to contain the correct goal letter. When that goal letter is inserted, its Scrabble value must be paid. Any extra changes made along the path would only add more cost. Therefore, summing the values of the goal letters that still need to be inserted cannot overestimate the optimal remaining cost.

For frequency, $h(n)$ is the number of positions where the current word and goal differ.

Every frequency-cost action costs $1 + \text{rarity}(w)$, where rarity is nonnegative. This means every action costs at least 1. Since one action can only correct one position, a state with k mismatched positions should need at least k actions. Its remaining frequency cost must therefore be at least k , so this heuristic is admissible.

2 Question 2: Does the Heuristic Actually Help?

I ran uniform-cost search and A* on both required word pairs using all three cost measures. I used a new DC problem instance for each search.

Word Pair	Cost	UCS Cost	UCS Nodes	A* Cost	A* Nodes	Reduction
WHY → ASK	Steps	5.00	773	5.00	16	97.93%
WHY → ASK	Scrabble	9.00	765	9.00	13	98.30%
WHY → ASK	Frequency	38.86	808	38.86	755	6.56%
BEAR → DUCK	Steps	4.00	691	4.00	6	99.13%
BEAR → DUCK	Scrabble	11.00	2981	11.00	6	99.80%
BEAR → DUCK	Frequency	38.24	1084	38.24	720	33.58%

UCS and A* returned the same optimal cost in all six tests, and the paths found can be different, but the total optimal cost was the same in every case.

For WHY → ASK, the Scrabble heuristic gave the largest reduction in expanded nodes at 98.30%.

For BEAR → DUCK, the Scrabble heuristic also gave the largest reduction at 99.80%.

For both word pairs, the frequency heuristic was the least informative. It only reduced the number of expanded nodes by 6.56% for WHY → ASK and 33.58% for BEAR → DUCK.

The steps heuristic works well because the number of mismatched positions gives a good estimate of how many letter changes are still required.

The Scrabble heuristic is more informative because it also uses the actual Scrabble values of the goal letters that still need to be inserted.

The frequency heuristic is less informative because it only counts mismatched positions and does not use the rarity values of the words that may be entered later. Two states can have the same number of mismatched letters but very different remaining frequency costs, so A* gets less useful information from this heuristic.

3 Question 3: Reversibility and Direction-Dependent Costs

For steps: Yes. Every action costs 1, so reversing a path gives a path with the same number of actions and the same total cost.

For Scrabble: No. The cost depends on the letter being inserted. For example, dog → cog costs 3 because c is inserted, while cog → dog costs 2 because d is inserted.

For frequency: No. The cost depends on the word being entered. Since dog has rarity 5.819885 and cog has rarity 11.393166, dog → cog costs 12.393166, while cog → dog costs 6.819885.

The actions are reversible, but the Scrabble and frequency costs are not symmetric.

4 Question 4: LLM Heuristic Audit

I used Chatgpt 5.6.

The exact prompt given to the model was:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

The model suggested adding the rarity of the goal word to the mismatch heuristic for every state that is not already the goal.

If the current state is the goal, the heuristic returns 0. Otherwise, it returns the number of mismatched positions plus the rarity of the goal word.

In pseudocode:

```
if state == goal: return 0
```

```
count = number of mismatched positions return count + dictionary[goal]
```

I determined that this heuristic is admissible.

If there are k mismatched positions, at least k actions are required because one action changes only one position. Each of those actions contributes at least 1 to the frequency cost.

Also, every solution must eventually enter the goal word. Entering the goal has a cost that includes the rarity of the goal. Therefore, every solution from a non-goal state has a cost of at least $k + \text{rarity}(\text{goal})$

The heuristic is also at least as large as h_0 . In the supplied dictionary the rarity values are positive, so it is strictly larger than h_0 for non-goal states.

Because the proposed heuristic is already admissible, I did not need to repair it.

I compared the original frequency heuristic h_0 with the proposed rarity-aware heuristic h_1 on the two frequency problems from Question 2.

Both heuristics returned the same optimal solution costs.

For WHY $\rightarrow$ ASK, h_0 returned cost 38.86 and expanded 755 nodes, while h_1 returned cost 38.86 and expanded 634 nodes.

For BEAR $\rightarrow$ DUCK, h_0 returned cost 38.24 and expanded 720 nodes, while h_1 returned cost 38.24 and expanded 302 nodes.

The new heuristic expanded fewer nodes on both problems while still finding the same optimal costs, so these results support using it instead of h_0 .